# Übungsblatt 2: RISC-V-Registerbefehle

**Mikrocomputertechnik 1**

## Aufgabe 1: Das Korrekturprogramm von Hand ausführen (8 Punkte)

Ein Drucksensor liefert ganzzahlige Rohwerte. Ein kurzes RV32I-Programm korrigiert sie nach $Y = (A - B) + 4C$: $A$ ist der Rohwert, $B$ der Nullpunkt des Sensors und $C$ eine Temperaturstufe, die je Stufe vier Zählwerte ausgleicht. Das Programm begleitet Sie durch das ganze Blatt. Es beginnt nach Lehrannahme bei Adresse `0x1000`; ohne komprimierte Befehle ist jeder native Befehl vier Byte lang.

```
addi t0, x0, 12      # A: Rohwert
addi t1, x0, 19      # B: Nullpunkt
addi t2, x0, 5       # C: Temperaturstufe
sub  t3, t0, t1      # A - B
slli t4, t2, 2       # 4C durch Schieben
add  t5, t3, t4      # Y
```

1. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (5 P)

| Adresse | Befehl | gelesene Werte | Ergebnis | Wirkung |
|---|---|---|---|---|
| `0x1000` | `addi t0, x0, 12` | 0 und Konstante 12 | 12 | t0 = 12 |
| | `addi t1, x0, 19` | | | |
| | `addi t2, x0, 5` | | | |
| | `sub t3, t0, t1` | | | |
| | `slli t4, t2, 2` | | | |
| | `add t5, t3, t4` | | | |

2. Geben Sie t3 als 32-Bit-Hexwort an. Unter welchen Architekturnamen `x...` finden Sie t3 und t5 in einer Registeranzeige? (1 P)
3. Welchen Wert hat der PC nach dem letzten Befehl, und wie viele Byte belegt das Programm? (1 P)
4. Um ein Register zu sparen, schreibt jemand in Zeile 4 `sub t0, t0, t1` und in Zeile 6 `add t5, t0, t4`. Welchen Wert hat t0 danach? Stimmt Y noch, und welche Eingabe ist anschließend verloren? (1 P)

## Aufgabe 2: Statuswort und Messzähler (10 Punkte)

Neben dem Messwert liefert der Sensor ein Statuswort, hier t0 = `0x0000004D`; die unteren acht Bit sind $0100\,1101_2$. Bit 0 meldet „Messwert gültig“, Bit 1 „Überlast“ und Bit 7 „Selbsttest läuft“. Logische Immediates werden wie bei addi von zwölf auf 32 Bit vorzeichenerweitert.

1. Ergänzen Sie Maske und Ergebnis. Jede Zeile startet mit t0 = `0x0000004D`. Die erste Zeile ist ein Beispiel. Tipp: Die Maske ist das vorzeichenerweiterte Immediate. (5 P)

| Befehl | Zweck | Maske (32 Bit) | Ergebnis |
|---|---|---|---|
| `andi t1, t0, 0x0F` | untere vier Bit behalten | `0x0000000F` | `0x0000000D` |
| `andi t2, t0, 1` | Bit 0 prüfen | | |
| `ori t3, t0, 0x80` | Bit 7 setzen | | |
| `xori t4, t0, 2` | Bit 1 kippen | | |
| `andi t5, t0, -16` | untere vier Bit löschen | | |
| `xori t6, t0, -1` | alle Bit invertieren | | |

2. Die unteren zwölf Bit des Statusworts sollen erhalten bleiben. Warum ist `andi t1, t0, 4095` keine gültige Instruktion? Geben Sie zwei Befehle an, die die Aufgabe lösen. (2 P)

Ein 32-Bit-Register zählt die Messungen; jede Messung führt `addi t0, t0, 1` aus.

3. Ergänzen Sie die Tabelle. Die erste Zeile ist ein Beispiel. (2 P)

| Zählerstand vorher | neues Wort | signed gelesen | vorzeichenlos gelesen |
|---|---|---|---|
| `0x00000009` | `0x0000000A` | 10 | 10 |
| `0x7FFFFFFF` | | | |
| `0xFFFFFFFF` | | | |

4. Nach 2 147 483 647 Messungen zeigt die Anzeige plötzlich −2 147 483 648. Hat der Prozessor einen Fehler gemeldet? Welche Lesart hätte die Anzeige verwenden müssen? (1 P)

## Aufgabe 3: Mit Schieben multiplizieren und mitteln (8 Punkte)

Zeile 5 des Korrekturprogramms bildet $4C$ durch Linksschieben, denn Multiplikationsbefehle gehören zur Erweiterung M, nicht zu RV32I. Außerdem mittelt das Gerät vier korrigierte Werte: Ihre Summe steht in t6, und `srai t6, t6, 2` teilt sie durch vier.

1. Ergänzen Sie die Tabelle. Jede Zeile startet unabhängig. Die erste Zeile ist ein Beispiel. (4 P)

| Befehl | Quellen vorher | Ergebnis (hex) | dezimal |
|---|---|---|---|
| `slli t4, t2, 2` | t2 = 5 | `0x00000014` | 20 |
| `slli t4, t2, 2` | t2 = −2 = `0xFFFFFFFE` | | |
| `srai t6, t6, 2` | t6 = 52 = `0x00000034` | | |
| `srai t6, t6, 2` | t6 = −28 = `0xFFFFFFE4` | | |
| `sll t4, t2, t5` | t2 = 5, t5 = 34 | | |

2. Ein anderes Gerät zeigt bei der Summe −28 den Mittelwert 1 073 741 817 an; bei positiven Summen stimmt der Wert. Welcher Befehl steht dort vermutlich statt `srai`? Begründen Sie mit dem Bitmuster. (2 P)
3. Die vier Werte −7, −8, −7 und −8 ergeben die Summe −30 = `0xFFFFFFE2`. Welchen Wert liefert `srai t6, t6, 2`? Wie liegt er zum mathematischen Mittelwert −7,5? (1 P)
4. Mit t2 = 5 und t5 = 32 liefert `sll t4, t2, t5` den Wert 5. Warum? (1 P)

## Aufgabe 4: Konstanten im Befehl (8 Punkte)

Die Zeilen 1 bis 3 des Korrekturprogramms setzen die Eingaben mit addi. Das Immediate-Feld ist zwölf Bit breit und signed: Es reicht von −2048 bis 2047. Eine direkt angegebene Shiftweite reicht von 0 bis 31.

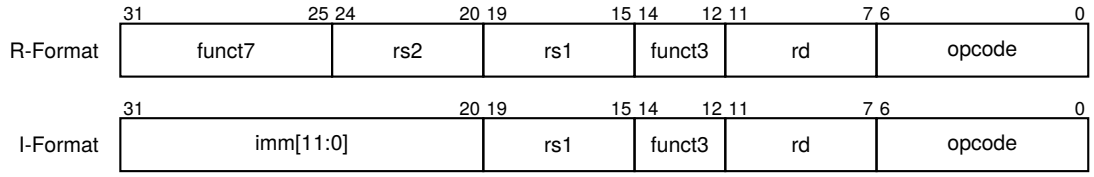
1. Ist die Zeile eine native Instruktion, eine Pseudoinstruktion oder unzulässig? Begründen Sie kurz. Die erste Zeile ist ein Beispiel. (3 P)

| Zeile | Art | Begründung |
|---|---|---|
| `addi t1, x0, 19` | nativ | 19 liegt im Bereich −2048 bis 2047 |
| `addi t1, x0, -2048` | | |
| `addi t1, x0, 2048` | | |
| `addi t3, t0, -19` | | |
| `li t1, 3000` | | |
| `mv t3, t0` | | |
| `slli t4, t2, 32` | | |

2. Weil der Nullpunkt fest 19 ist, kann `addi t3, t0, -19` die Zeilen 2 und 4 ersetzen. Geben Sie −19 als Zwölf-Bit-Muster, als drei Hexziffern und als vorzeichenerweitertes 32-Bit-Wort an. (1 P)
3. Für welche der Konstanten $K = 19$, $K = 2048$ und $K = -2048$ lässt sich t0 $- K$ mit einem einzigen addi berechnen? (1 P)
4. Ein neuer Sensor hat den Nullpunkt $B = 3000$. Eine gültige native Entwicklung von `li t1, 3000` ist `lui t1, 0x1` gefolgt von `addi t1, t1, k`. Welchen Wert hat t1 nach lui, und welches k ist nötig? Tipp: lui setzt die Bits 31 bis 12 und löscht die unteren zwölf Bit. (2 P)
5. Zeile 2 des Programms aus Aufgabe 1 lautet nun `li t1, 3000`. Wie viele Byte belegt das Programm jetzt, und an welcher Adresse steht sub? (1 P)

## Aufgabe 5: Das Korrekturprogramm codieren (14 Punkte)

Die Felder stehen von Bit 31 links bis Bit 0 rechts. Registerfelder enthalten Nummern: x0 = 0, t0 bis t2 = x5 bis x7, t3 bis t6 = x28 bis x31.




| Befehl | opcode | funct3 | funct7 |
|---|---|---|---|
| add | 0110011 | 000 | 0000000 |
| sub | 0110011 | 000 | 0100000 |
| addi | 0010011 | 000 | — |

1. Ergänzen Sie die Felder von `sub t3, t0, t1`. Die erste Zeile ist ein Beispiel. (2 P)

| Befehl | funct7 | rs2 | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|---|---|
| `add t5, t3, t4` | 0000000 | 11101 | 11100 | 000 | 11110 | 0110011 |
| `sub t3, t0, t1` | | | | | | |

2. Schreiben Sie das Wort von `sub t3, t0, t1` in Vierergruppen und als Hexwort. Zum Vergleich ergibt das Beispiel `0000 0001 1101 1110 0000 1111 0011 0011` = `0x01DE0F33`. (2 P)
3. add und sub haben denselben opcode. Welches Feld unterscheidet sie, und welches Instruktionsbit ändert sich dabei? (1 P)

4. Ergänzen Sie die Felder des I-Formats und das Hexwort. Die erste Spalte ist ein Beispiel. (4 P)

| Feld | addi t0, x0, 12 | addi t1, x0, 19 | addi t3, t0, −19 |
|---|---|---|---|
| imm[11:0] | 000000001100 | | |
| rs1 | 00000 | | |
| funct3 | 000 | | |
| rd | 00101 | | |
| opcode | 0010011 | | |
| Hexwort | 0x00C00293 | | |

5. Lesen Sie aus Ihrem Hexwort für `addi t3, t0, -19` die oberen drei Hexziffern als signed Zwölf-Bit-Zahl. Erhalten Sie −19 zurück? (1 P)
6. Für eine neue Messung ist der Rohwert $A = 30$. Welche Wörter des Programms aus Aufgabe 1 ändern sich? Warum bleibt das Wort von sub gleich? (2 P)
7. Das Wort von `sub t3, t0, t1` liegt ab Adresse `0x100C`. Ergänzen Sie die Bytes in Little Endian. (2 P)

| Adresse | `0x100C` | `0x100D` | `0x100E` | `0x100F` |
|---|---|---|---|---|
| Byte | | | | |

## Aufgabe 6: Fehlersuche bei falschen Korrekturwerten (12 Punkte)

Drei Geräte sollen das Programm aus Aufgabe 1 mit $A = 12$, $B = 19$ und $C = 5$ ausführen. Richtig wäre $Y = 13$.

**Gerät 1 meldet $Y = 27$.**

1. Angenommen, t4 = 20 stimmt. Welcher Wert stand dann in t3? Wie kann er aus 12 und 19 entstanden sein? (2 P)
2. Ein Speicherabzug zeigt an den Adressen `0x100C` bis `0x100F` die Bytes `33`, `0E`, `53`, `40`. Setzen Sie das 32-Bit-Wort zusammen. (1 P)
3. Zerlegen Sie das Wort in die Felder des R-Formats und geben Sie die Registernummern an. (3 P)

| funct7 | rs2 | rs1 | funct3 | rd | opcode |
|---|---|---|---|---|---|
| | | | | | |

4. Welcher Befehl steht dort, geschrieben mit Aliasnamen? Bestätigt das Ihre Vermutung aus Teil 1? Welche Bytes müssten ab `0x100C` stehen? (2 P)

**Gerät 2 meldet $Y = 33$.**

5. Angenommen, t3 = −7 stimmt. Welcher Wert stand in t4? Welcher Befehl wurde vermutlich falsch eingegeben, und wie lautet er? (2 P)

**Gerät 3 soll den neuen Sensor mit dem Nullpunkt 3000 bedienen.** Jemand hat Zeile 2 als `addi t1, x0, 3000` geschrieben; die Übersetzung scheitert.

6. Warum? Wie schreiben Sie die Zeile, damit t1 = 3000 entsteht? (2 P)

## Aufgabe 7: Die Messkorrektur in Venus prüfen

Ergänzen Sie den folgenden Programmrahmen. `li` setzt die Eingaben; bei Werten von −2048 bis 2047 entsteht daraus ein einzelnes addi.

```
.text
.globl main
main:
    li   t0, 12          # A: Rohwert
    li   t1, 19          # B: Nullpunkt
    li   t2, 5           # C: Temperaturstufe
    # Lücke 1: drei Befehle, Ergebnis Y in t5
Ergebnis_pruefen:
    # Lücke 2: Y als Ganzzahl ausgeben
    # Lücke 3: Programm beenden
```

Venus-Dienste der Kursumgebung: a0 enthält die Dienstnummer, danach folgt `ecall`. Dienst 1 gibt die Ganzzahl in a1 aus, Dienst 10 beendet das Programm.

1. Füllen Sie die Lücken. Lücke 1 übernehmen Sie aus Aufgabe 1, für Lücke 2 und 3 genügen fünf Befehle.
2. Öffnen Sie Venus unter thaumicmekanism.github.io/venus und geben Sie das Programm im Tab *Editor* ein. Der Wechsel auf den Tab *Simulator* assembliert es. Die Programmansicht zeigt die Spalten *Machine Code* (Hexwort), *Basic Code* (native Befehle nach der Entwicklung) und *Original Code*. Notieren Sie die Wörter von `li t0, 12` und von sub und vergleichen Sie mit Aufgabe 5.
3. Ein Klick auf einen Befehl setzt einen Haltepunkt (rote Zeile). Setzen Sie ihn auf den ersten Befehl nach `Ergebnis_pruefen` und starten Sie mit *Run*. Alternativ führt *Step* den grün markierten Befehl aus, *Prev* nimmt den letzten Schritt zurück. Prüfen Sie in der Registeransicht t3 bis t5 (x28 bis x30). Werte erscheinen standardmäßig vorzeichenlos hexadezimal, die Anzeigeeinstellung *Decimal* zeigt sie signed dezimal.
4. Führen Sie das Programm zu Ende und notieren Sie die Konsolenausgabe.
5. Sagen Sie für jede Zeile t3, t4 und Y voraus. Ändern Sie dann die drei li-Zeilen, assemblieren Sie neu und prüfen Sie. *Reset* setzt den Simulator auf den Anfangszustand zurück.

| A | B | C | t3 | t4 | Y |
|---|---|---|---|---|---|
| 12 | 19 | 5 | −7 | 20 | 13 |
| 0 | 0 | 0 | | | |
| −8 | 7 | 2 | | | |
| 9 | −4 | −2 | | | |
| 3012 | 3000 | 5 | | | |

6. Wie viele native Befehle zeigt die Spalte *Basic Code* in der letzten Variante für die drei li-Zeilen? Vergleichen Sie mit Aufgabe 4. Ersetzen Sie testweise `li t1, 3000` durch `addi t1, x0, 3000` und notieren Sie die Meldung.

Protokollieren Sie jeweils Vorhersage, Registeranzeige am Haltepunkt und Konsolenausgabe.